JARVIS VOICE ASSISTANT

Comprehensive Technical Documentation & Architectural Blueprint

1. Executive Summary

JARVIS is a 100% offline, privacy-focused, local desktop voice assistant designed for Windows OS. It operates continuously in the background, listening for the custom wake word *'Hey Jarvis'*, transcribing speech locally using GPU acceleration, generating intelligent responses via a local LLM, and presenting real-time visual feedback through a native PyQt6 desktop interface and system tray application.

Core Metric	Specification / Technology	Implementation Details
Wake Word Engine	openWakeWord (ONNX Runtime)	16kHz mono, model: *hey_jarvis*
Speech-to-Text (STT)	Faster-Whisper (base model)	CUDA GPU FP16 execution, CPU int8 fallback
Large Language Model	Ollama (llama3.1:8b)	Local HTTP REST API, concise 1-3 sentence system prompt
Text-to-Speech (TTS)	pyttsx3 (Windows SAPI5)	Offline native Windows COM voice synthesis
User Interface	PyQt6 Native Desktop GUI	Animated Arc-Reactor HUD, dark glassmorphic layout
Audio Processing	sounddevice & NumPy	Energy RMS VAD, 10x digital gain multiplier

2. Technology Stack Breakdown & Architectural Rationale

Every library and component in Jarvis was chosen strategically to fulfill three non-negotiable requirements: **100% Local Privacy**, **Zero Cloud Subscription Costs**, and **Low Latency Real-time Performance**.

• openWakeWord & ONNX Runtime — *Continuous Wake Word Detection ('Hey Jarvis')*

Uses lightweight neural models running via ONNX Runtime. It uses less than 1% CPU while continuously monitoring the microphone stream. Unlike cloud wake-word engines (Alexa/Siri), it requires zero internet connectivity.

• Faster-Whisper (CTranslate2) — *Local Speech-to-Text Transcription*

Faster-Whisper is a reimplementation of OpenAI's Whisper model using CTranslate2. It is up to 4x faster than standard PyTorch Whisper while consuming 50% less VRAM. Configured for CUDA FP16 GPU acceleration, it transcribes speech in under 0.5 seconds.

- **Ollama & llama3.1:8b — *Local Large Language Model Intelligence***

  Ollama serves Meta's 8B parameter model locally via a REST API on port 11434. This eliminates API costs (like OpenAI GPT-4) and guarantees total conversation privacy. The prompt is engineering-constrained to brief, conversational 1-3 sentence answers optimized for voice.

- **PyQt6 (Qt for Python) — *Native Windows Desktop GUI & System Tray***

  Provides a native C++-backed Windows application window. Features a custom QPainter animated Arc-Reactor HUD widget, dark glassmorphism stylesheet, thread-safe pyqtSignal event bridges, and automatic minimize-to-system-tray background operation.

- **sounddevice & NumPy — *Real-time Audio Stream & VAD Processing***

  Reads 16kHz PCM audio buffers directly from PortAudio. Implements energy-based Root-Mean-Square (RMS) Voice Activity Detection (VAD) to auto-detect when speech ends. Features a custom 10x digital software gain multiplier to fix low-gain laptop microphones.

- **pyttsx3 (SAPI5) — *Offline Voice Synthesis (Text-to-Speech)***

  Interfaces directly with Windows SAPI5 COM engine. Provides instant voice output without cloud API latency or web network dependencies.

3. Technical Challenges & Engineering Solutions

Challenge 1: Windows Laptop Microphone Low-Gain (2% Max Signal Level)

Issue: On modern Windows laptops (specifically Intel® Smart Sound Technology Digital Mic Arrays), hardware volume sliders are locked at 100% without a hardware boost (+10dB/+20dB) option. The microphone captured peak amplitudes of barely 2% (~20 units out of 32,767), causing the wake-word engine to report *Microphone signal level is near ZERO*.
Solution: Implemented a 10x digital software gain multiplier (AUDIO_GAIN = 10.0) inside jarvis.py. Audio PCM arrays are multiplied by 10.0 and clipped to int16/float32 bounds before VAD and wake-word evaluation, amplifying quiet voice signals to normal operating levels.

Challenge 2: Ollama First-Run Cold-Start Timeout (30s Exceeded)

Issue: On the first query after startup, Ollama requires ~32-35 seconds to load the 4.9 GB llama3.1:8b weights from disk into GPU VRAM. The initial HTTP request timed out due to a 30-second timeout=30 limit.
Solution: Increased HTTP request timeout to 120 seconds (timeout=120) in jarvis.py and added dedicated requests.exceptions.Timeout exception handling. Subsequent queries execute in 1-3 seconds once the model is warmed up in VRAM.

Challenge 3: Multi-threaded GUI Responsiveness & System Tray Mode

Issue: Audio capture and LLM network requests are blocking operations. Running them on the GUI thread causes the application window to freeze and crash.
Solution: Architected a clean multi-threaded design. The PyQt6 QApplication runs on the main GUI thread while JarvisAssistant runs on a background QThread (AssistantWorkerThread). Thread-safe pyqtSignal events bridge state changes, user transcripts, and AI responses safely to the UI. Integrated QSystemTrayIcon for background minimization.

4. End-to-End Execution Flow

Stage	Action	Data Output
1. Passive Listening	Microphone audio stream read in 1280-sample chunks (~80ms) via sounddevice with 10x gain.	PCM int16 Array
2. Wake Detection	openWakeWord ONNX engine evaluates chunk. If confidence score >= 0.5, triggers active state.	State -> RECORDING
3. Voice Recording	Energy RMS VAD listens to user voice until 1.5 seconds of silence is detected.	Float32 Audio Array
4. Transcription	Faster-Whisper (CUDA FP16) transcribes audio into text.	User Transcript String
5. LLM Inference	Transcript sent to Ollama llama3.1:8b local API with concise persona prompt.	LLM Response Text
6. Voice & GUI Output	pyttsx3 speaks text out loud; PyQt6 GUI updates Arc-Reactor & chat bubble.	Audio Speech & UI Render

5. Operating & Launch Instructions

Launch Desktop GUI Window:

```
python jarvis_gui.py
```

Background Execution:

Closing or minimizing the window automatically keeps Jarvis active in the Windows System Tray (next to the clock). Say *'Hey Jarvis'* anytime or double-click the tray icon to re-open the desktop interface.